ActionEvent object [e.getActionCommand()] it returns a String. If we want to make a comparison between that command String, we can't just compare the two, we have to use the equals() method, which is inherited from Object.

The JButton object lets you specify a different "rollover" icon—when a mouse pointer is on top of the button:

You can set a JButton to be the Default Button, meaning it automatically has the focus. To do this, you have to get an object of the JRootPane, which we have not yet had to do:

```
Container c = getContentPane();
JRootPane root = getRootPane();
JButton def = new JButton( "Default Button" );
root.setDefaultButton( def );
c.add( def );
```

Using a method called doClick(), you can make a button be clicked programmatically—meaning seemingly by magic. The argument is the time in milliseconds that the button will remain pressed.

```
JButton click = new JButton( "Click" );
click.doClick( 2000 );
```

10.7 JCheckBox

These are two-state buttons that reflect their status by having a checkmark in a box (as the name implies). We say they can be Selected or DeSelected.

Constructors are many and varied. It can include a String, an Icon and it can be preset as either selected or deselected. Event Handling for a JCheckBox is different—it uses an ItemListener rather than an ActionListener. Your program must implement the ItemListener interface. This interface has only one method for you to override:

```
public void itemStateChanged( ItemEvent e );
```

An ItemEvent object is passed, and it contains a constant property: ItemEvent.SELECTED

Method getStateChange() tells what happened.

```
JCheckBox ans = new JCheckBox( "Yes" );
ans.addItemListener( this );
…
public void itemStateChanged( ItemEvent e )
{
if( e.getStateChange() == ItemEvent.SELECTED )
}
```